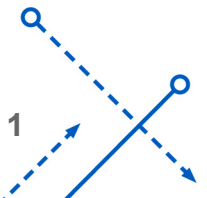


CSE 305 : Functional Programming in OCaml continued

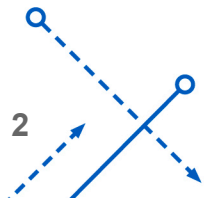
(slides from David Walker, Princeton,
Modified by LZ)

Lukasz Ziarek
Office: Davis Hall 338E
lziarek@buffalo.edu



Learning Goals for today

- Types and Functions
- Anonymous Functions
- First Class Functions
- Map / Reduce



Rules for type-checking functions

General Rule:

If a function $f: T1 \rightarrow T2$
and an argument $e: T1$
then $f\ e: T2$

Example:

```
add_one : int -> int
```

```
3 + 4 : int
```

```
add_one (3 + 4) : int
```

Rules for type-checking functions

- Recall the type of add:

Definition:

```
let add (x:int) (y:int) : int =  
  x + y
```

Type:

```
add : int -> int -> int
```

Rules for type checking functions

- Recall the type of add:

Definition:

```
let add (x:int) (y:int) : int =  
  x + y
```

Type:

```
add : int -> int -> int
```

Same as:

```
add : int -> (int -> int)
```

Rules for type checking functions

General Rule:

If a function $f: T1 \rightarrow T2$
and an argument $e: T1$
then $f\ e: T2$

$A \rightarrow B \rightarrow C$

same as:

$A \rightarrow (B \rightarrow C)$

Example:

```
add : int -> int -> int
```

```
3 + 4 : int
```

```
add (3 + 4) : ???
```

Rules for type checking functions

General Rule:

If a function $f: T1 \rightarrow T2$
and an argument $e: T1$
then $f\ e: T2$

$A \rightarrow B \rightarrow C$

same as:

$A \rightarrow (B \rightarrow C)$

Example:

```
add : int -> (int -> int)
```

```
3 + 4 : int
```

```
add (3 + 4) :
```

Rules for type checking functions

General Rule:

If a function $f: T1 \rightarrow T2$
and an argument $e: T1$
then $f\ e: T2$

$A \rightarrow B \rightarrow C$

same as:

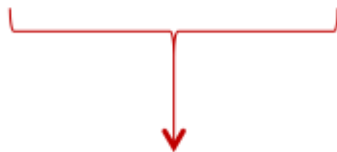
$A \rightarrow (B \rightarrow C)$

Example:

`add : int -> (int -> int)`

`3 + 4 : int`

`add (3 + 4) : int -> int`



Rules for type checking functions

General Rule:

If a function $f: T1 \rightarrow T2$
and an argument $e: T1$
then $f\ e: T2$

$A \rightarrow B \rightarrow C$

same as:

$A \rightarrow (B \rightarrow C)$

Example:

`add : int -> int -> int`

`3 + 4 : int`

`add (3 + 4) : int -> int`

`(add (3 + 4)) 7 : int`



Rules for type checking functions

General Rule:

If a function $f: T1 \rightarrow T2$
and an argument $e: T1$
then $f\ e: T2$

$A \rightarrow B \rightarrow C$

same as:

$A \rightarrow (B \rightarrow C)$

Example:

```
add : int -> int -> int
```

```
3 + 4 : int
```

```
add (3 + 4) : int -> int
```

```
add (3 + 4) 7 : int
```

Rules for type checking functions

Example:

```
let munge (b:bool) (x:int) : ?? =  
  if not b then  
    string_of_int x  
  else  
    "hello"  
;;  
  
let y = 17;;
```

```
munge (y > 17) : ??  
  
munge true (f (munge false 3)) : ??  
  f : ??  
  
munge true (g munge) : ??  
  g : ??
```

Rules for type checking functions

Example:

```
let munge (b:bool) (x:int) : ?? =  
  if not b then  
    string_of_int x  
  else  
    "hello"  
;;  
  
let y = 17;;
```



string

```
munge (y > 17) : ??  
  
munge true (f (munge false 3)) : ??  
  f : ??  
  
munge true (g munge) : ??  
  g : ??
```

Rules for type checking functions

Example:

```
let munge (b:bool) (x:int) : ?? =  
  if not b then  
    string_of_int x  
  else  
    "hello"  
;;  
  
let y = 17;;
```

string



```
munge (y > 17) : ??  
  
munge true (f (munge false 3)) : ??  
  f : ??  
  
munge true (g munge) : ??  
  g : ??
```

Rules for type checking functions

Example:

```
let munge (b:bool) (x:int) : ?? =
  if not b then
    string_of_int x
  else
    "hello"
;;
let y = 17;;
```

`munge: bool -> int -> string`

```
munge (y > 17) : ??

munge true (f (munge false 3)) : ??
  f : ??

munge true (g munge) : ??
  g : ??
```

Rules for type checking functions

Example:

```
let munge (b:bool) (x:int) : ?? =  
  if not b then  
    string_of_int x  
  else  
    "hello"  
;;
```

`munge: bool -> int -> string`

```
let y = 17;;
```

```
munge (y > 17) : ??
```

```
munge true (f (munge false 3)) : ??  
f : ??
```

```
munge true (g munge) : ??  
g : ??
```

Rules for type checking functions

Example:

```
let munge (b:bool) (x:int) : ?? =  
  if not b then  
    string_of_int x  
  else  
    "hello"  
;;  
  
let y = 17;;
```

```
munge (y > 17) : ??  
  
munge true (f (munge false 3)) : ??  
  f : string -> int  
  
munge true (g munge) : ??  
  g : (bool -> int -> string) -> int
```

One key thing to remember

- If you have a function f with a type like this:

$A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow F$

- Then each time you add an argument, you can get the type of the result by knocking off the first type in the series

$f\ a1 : B \rightarrow C \rightarrow D \rightarrow E \rightarrow F$ (if $a1 : A$)

$f\ a1\ a2 : C \rightarrow D \rightarrow E \rightarrow F$ (if $a2 : B$)

$f\ a1\ a2\ a3 : D \rightarrow E \rightarrow F$ (if $a3 : C$)

$f\ a1\ a2\ a3\ a4\ a5 : F$ (if $a4 : D$ and $a5 : E$)

More functional programming

Another example

```
let rec sum (xs:int list) : int =  
  match xs with  
    | [] -> 0  
    | hd::tl -> hd + (sum tl)
```

```
let rec prod (xs:int list) : int =  
  match xs with  
    | [] -> 1  
    | hd::tl -> hd * (prod tl)
```

Goal: Create a function called **reduce** that when supplied with a few arguments can implement both sum and prod. Define sum2 and prod2 using reduce.

(Try it)

Another example

```
let rec sum (xs:int list) : int =  
  match xs with  
    | [] -> b  
    | hd::tl -> hd + (sum tl)
```

```
let rec prod (xs:int list) : int =  
  match xs with  
    | [] -> b  
    | hd::tl -> hd * (prod tl)
```

Another example

```
let rec sum (xs:int list) : int =  
  match xs with  
    | [] -> b  
    | hd::tl -> hd OP (RECURSIVE CALL ON tl)
```

```
let rec prod (xs:int list) : int =  
  match xs with  
    | [] -> b  
    | hd::tl -> hd OP (RECURSIVE CALL ON tl)
```

Another example

```
let rec sum (xs:int list) : int =  
  match xs with  
    | [] -> b  
    | hd::tl -> f hd (RECURSIVE CALL ON tl)
```

```
let rec prod (xs:int list) : int =  
  match xs with  
    | [] -> b  
    | hd::tl -> f hd (RECURSIVE CALL ON tl)
```

A generic reducer

```
let add x y = x + y
```

```
let mul x y = x * y
```

```
let rec reduce (f:int->int->int) (b:int) (xs:int list) : int =  
  match xs with  
    | [] -> b  
    | hd::tl -> f hd (reduce f b tl)
```

```
let sum xs = reduce add 0 xs
```

```
let prod xs = reduce mul 1 xs
```

Using anonymous functions

```
let rec reduce (f:int->int->int) (b:int) (xs:int list) : int =  
  match xs with  
    | [] -> b  
    | hd::tl -> f hd (reduce f b tl)  
  
let sum xs = reduce (fun x y -> x+y) 0 xs  
let prod xs = reduce (fun x y -> x*y) 1 xs
```

Using anonymous functions

```
let rec reduce (f:int->int->int) (b:int) (xs:int list) : int =  
  match xs with  
    | [] -> b  
    | hd::tl -> f hd (reduce f b tl)  
  
let sum xs = reduce (fun x y -> x+y) 0 xs  
let prod xs = reduce (fun x y -> x*y) 1 xs  
  
let sum_of_squares xs = sum (map (fun x -> x * x) xs)  
let pairify xs = map (fun x -> (x,x)) xs
```

What's going on under the hood?

Anonymous functions

Function declarations:

```
let square x = x*x  
let add x y = x+y
```

Are **syntactic sugar** for:

```
let square = (fun x -> x*x)  
let add = (fun x y -> x+y)
```

In other words, ***functions are values*** we can bind to a variable, just like 3 or “moo” or true.

Functions are 2nd class no more!

One argument, one result

Simplifying further:

```
let add = (fun x y -> x+y)
```

is shorthand for:

```
let add = (fun x -> (fun y -> x+y))
```

That is, add is a **function** which:

- when given a value x, *returns a function* (fun y -> x+y) which:
 - when given a value y, returns x+y.

Curried functions

Currying: verb. gerund or present participle

(1) to prepare or flavor with hot-tasting spices

(2) To encode a multi-argument function using nested, higher-order functions

(1)



(2)

```
fun x -> (fun y -> x+y) (* curried *)  
fun x y -> x + y         (* curried *)  
fun (x,y) -> x+y         (* uncurried *)
```

What is the type of add?

```
let add = (fun x -> (fun y -> x+y))
```

Add's type is:

```
int -> (int -> int)
```

which we can write as:

```
int -> int -> int
```

That is, the arrow type is right-associative

What's so good about Currying?

In addition to simplifying the language, currying functions so that they only take one argument leads to two major wins:

1. We can *partially apply* a function.
2. We can more easily *compose* functions.



Partial application

```
let add = (fun x -> (fun y -> x+y))
```

Curried functions allow defs of new, partially applied functions:

```
let inc = add 1
```

Equivalent to writing:

```
let inc = (fun y -> 1+y)
```

which is equivalent to writing:

```
let inc y = 1+y
```

also:

```
let inc2 = add 2  
let inc3 = add 3
```